

PaymentProcessor.java - Code Structure & Links

1. CLASS DEFINITION

```
public class PaymentProcessor
```

Explanation: This is the Execution Engine. It is completely independent and does not inherit from anything. It links dynamically to the Payment class to process transactions without needing to know their specific types.

2. CLASS VARIABLES (FIELDS)

```
private List<Payment> queue = new ArrayList<>();  
private List<Payment> history = new ArrayList<>();
```

Explanation: These variables use Polymorphic Storage. Notice the type is List<Payment>, not List<CreditCardPayment>. Because all payment types link to the Payment base class, this single queue can hold every type of payment at the same time.

3. METHODS & ALGORITHMS

```
public void addPayment(Payment payment)
```

Explanation: Accepts any object that extends Payment and adds it to the queue.

```
public List<ProcessResult> processAll()
```

Explanation: The core algorithm demonstrating Dynamic Dispatch.

Algorithm Flow:

1. It loops through every generic "Payment" object in the queue.
2. It creates a lambda log function to capture text output.
3. It calls payment.process(log).

This is the magic: The JVM looks at the generic variable, identifies its true underlying subclass in memory (e.g., CreditCard vs PayPal), and executes that specific class's code automatically.

```
public double totalSuccessful()
```

Explanation: Uses Java 8 Streams to loop through the history list, filter out anything where getStatus() is not "SUCCESS", extract the amounts, and sum them up.